

目次

Chapter 1

コンピュータの重箱の隅	3
1-1 論理回路から加算器は飛躍しすぎ！？	4
1-1-1 プログラムを実行する手順を確認する	4
1-1-2 論理回路の誕生	5
1-2 通信機能は5大装置のどこに入る？	8
1-2-1 h3見出し（小見出し）	10
1-3 デジタルとアナログ	11
1-3-1 ルビサンプル	14
1-3-2 数式サンプル	16
1-3-3 会話サンプル	18

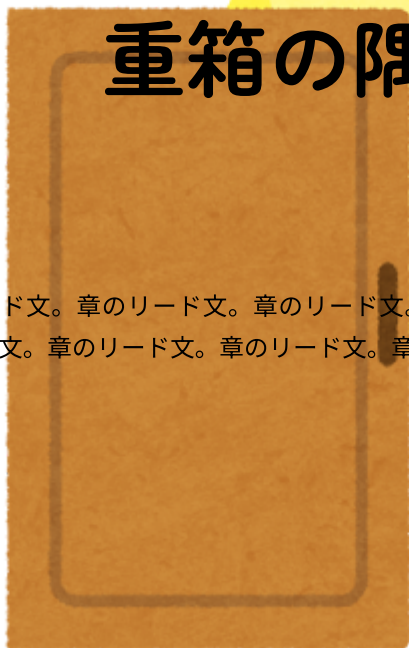
Chapter 2

コンピュータの重箱の隅	??
2-1 論理回路から加算器は飛躍しすぎ？	??
2-1-1 論理回路の誕生	??
2-2 通信機能は5大装置のどこに入る？	??
2-3 デジタルとアナログ	??
2-3-1 フォントテスト	??

chapter1

コンピュータの 重箱の隅

章のリード文。章のリード文。章のリード文。章のリード文。章
のリード文。章のリード文。章のリード文。章のリード文。



1-

論理回路から加算器 は飛躍しすぎ！？

ここでは、Rubyのプログラムを作成して、実行するまでの流れを解説します。最初にエディターでプログラムを書いて、コマンドプロンプトを実行するだけで、すぐに結果を確認できます。

◆ プログラムを実行する手順を確認する

Rubyプログラムを書いて実行してみましょう。Rubyプログラムを実行するには3つの手順を踏みます。

まずはプログラムファイルを保存するフォルダを作成しておきましょう。本書では、ファイルはC:\Users…過去に「コンピュータの歴史」を**ブール論理 (Boolean Logic)** の誕生あたりからたどろうとして、いろいろと調べてみたのだが、**ジョージ・ブール氏**の考えたものは現在ブール論理として習うものとかかなり違っていた。



① 「I accept for terms…」
にチェック

ブール氏が考えたのは、言葉で推論するとう**② 「Next」をクリック**じるので、数式を使って推論すべきだというものだった。実際、ブール氏の論文を見てみると、足し算や掛け算らしき数式しか書かれていない。

<https://www.gutenberg.org/ebooks/36884>にリンクしたい

現在使われている論理記号やAND、ORなどの用語は、後世に**数理論理学**として発展していく中で作られたもので、最初のとっかかりのアイ

デアを考えた人として、ブール氏の名前を冠することにしたい。

論理回路の誕生

ブール氏が論文を書いてから 100 年ほどして、電信に使われるリレーを使った**論理回路 (Logic Circuit)** が誕生した。**クロード・シャノン氏** が論理回路についての論文を書いたのが 1937 年で、ほぼ同時期に最初のデジタルコンピュータも誕生している。このあたり順番がはっきりしなくてモヤッとするが、たいていの発明・発見は、複数の人が同時期に同じ答えにたどりつくものようだ。

```
from pathlib import Path
from bs4 import BeautifulSoup

result = '## 目次{#toc role="doc-toc"}\n'
outpath = Path('tocoutput.md')
outpath.write_text(result, encoding='utf-8')
```

generate_toc.py

```
from pathlib import Path
from bs4 import BeautifulSoup

result = '## 目次{#toc role="doc-toc"}\n'
outpath = Path('tocoutput.md') -----コード引き出し線 (先頭が<)
outpath.write_text(result, encoding='utf-8')
```

ここで不思議なのは、スイッチを直列に並べて AND 回路、並列に並べて OR 回路が作れるというのは、ブール論理の延長として理解できる。しかし、その先、つまり**論理回路を使って計算する**という発想はどこからでてきたのだろう？



論理回路 (Logic Circuit)

2進数1桁の計算を行う**半加算器（Half Adder）**は、AND回路2つとOR回路1つ、NOT回路1つの組み合わせで作れる。

表見出し

●列見出し	●列見出し（中央ぞろえ）	●列見出し（右ぞろえ）
セル	セル	セル
セル	セル	90
セル	セルの中に長めの文を入れる	1000
●列見出し	●列見出し（中央ぞろえ）	●列見出し（右ぞろえ）
セル	セル	セル
セル	セル	90
セル	セルの中に長めの文を入れる	1000

Enter キー



半加算器（Half Adder）

半加算器（Half Adder）

しかし、これがブール論理の延長で、普通に出てくるとはちょっと考えにくい気がする。ブール論理は「AND（かつ）」「OR（または）」などで真偽を求めるものであって、四則演算するものではないはずだ。

残念ながら、加算器の発明者の資料は見つけられなかった。おそらく、何かを作ろうとして論理回路の組み合わせを考えているうちに、「0+0=00」「0+1=01」「1+0=01」「1+1=10」という結果を出せば足し算する回路が作れるという答えにパッとたどりついたのだろう。

世の中には頭のいい人はたくさんいるので、このぐらいは複数の人が同時に思いつくものなのかもしれない。

 論理回路 (Logic Circuit)

 半加算器 (Half Adder)

1-

通信機能は5大装置 のどこに入る？

コンピュータサイエンスの初歩として必ず登場するのが、**ノイマン型コンピュータ (von Neumann architecture)** もしくは**コンピュータの5大装置**だ。コンピュータは「制御」「演算」「記憶」「入力」「出力」という5つの装置で構成されるという説で、**フォン・ノイマン氏**がその論文を書いたので、ノイマン型と呼ぶ。ノイマン氏は悪魔のように頭がいい人だったが、5大装置をノイマン氏自身がゼロから考えたわけではなく、**EDVAC**という開発中のコンピュータを見学して、「つまり、こういうことだろ？」と論文をまとめたそうだ。

- **制御**：記憶装置からプログラムを読み込み、各装置に指示を出す
 - CPU
- **演算**：四則演算などの計算を行う
 - CPU
- **記憶**：プログラムやデータを記憶する
 - メモリ
 - HDD
 - SSD

- 入力：外部から情報を取り込む
 - キーボード
 - マウス
- 出力：外部に情報を書き出す
 - ディスプレイ
 - スピーカー



von Neumann architecture von Neumann architecture

今のパソコンやスマートフォンでも、5大装置という構成は変わらないので、今の教科書にもそのまま出てくる。ただ、5大装置の図に、実際のCPUやメモリ、キーボードなどの部品を当てはめていくと、ふと困ったことに気付く。通信モデムやNIC、タッチディスプレイなど、5つのカテゴリにうまく当てはまらない機器が結構出てくるのだ。

そのときはしかたなく、入力と出力を図の右側に配置して、「入出力兼用の機器もある」と説明したのだが、もっとうまいやり方があったのではと今でもときどき思う。

ノイマン氏に聞いてみたいところだが、悪魔のように頭のいい人だったらしいので、「馬鹿め、そのぐらい自分で考えろ」といわれそうだ。

◆h3見出し（小見出し）

●h4見出し（小小見出し）

h5見出し（コラム？）

h6見出し（図表見出し）

コラムタイトル

コラムテキスト。コラムタイトルのあとは1行空ける

1- デジタルとアナログ

よくコンピュータは**デジタル**だという。デジタルは「連続した量」を意味する**アナログ**の対義語で、連続していないという意味から日本語では「離散的な値」ともいう。電気や光、音といった自然現象はすべてアナログだ。

ここでよく考えてみると、コンピュータの電子回路も電気が流れているからアナログではないかという答えにたどり着く。実際のところ、コンピュータの電気信号も連続的に変化していて微妙なブレがあり、この範囲にだいたいおさまったら「1」、この範囲だったら「0」という感じに決めて、デジタルと見なしているそう。

digital analogdigital analog

自然界にはアナログしか存在せず、コンピュータもアナログ。ということは、純粋なデジタルは、人間の頭の中にしか存在しないということになる。加えていうとプログラム（ソフトウェア）も、人間のデジタル的な考え方を反映したものなので、デジタルといえる。

デジタルとアナログについて考えていくと面白いことがいろいろあって、昔のビデオテープや音楽テープはすべてアナログで、デジタル化し

たDVDやCDが登場したのは結構あとのことだ。これはなぜかという、デジタルのほうがデータ量が増えるからだ。

例えばアナログで「100」を表す場合、単位や媒体は何かはわからないが、100の強さの信号があればいい。デジタルで100を表す場合、2進数だと「01100100」となるので7ビット必要になる。仮に1秒間に1つしか信号を送れないとすると、アナログでは1秒で済むところが、デジタルでは7秒必要という計算になる。つまり、デジタルのほうがデータ量が増えるわけで、これを解決するには電子回路をはるかに高速にしなければならない。その発展に時間がかかったわけだ。



digital analog

digital analog

データ量が増える以外のデジタルの弱点として、連続して変化するアナログを、段階的なデジタルに変換する過程で、変化途中の細かな情報が切り捨てられるという点がある。アナログの信号（レコードとかフィルムとか）だったら、昔は読み取れなかった情報が未来の技術で読み取れる可能性があるが、デジタルの信号（DVDとかBDとか）はいつまで経っても記録時点のままなので、切り捨てられた情報は未来永劫復元できない。最近よく聞くデジタルリマスターも、大元のフィルムから起こすのであって、DVDを原本とすることはない。

などとききつつも、最近のAI技術では「存在しないものをそれらしく復元する」こともできてしまうので、将来的には何とかなってしまうかもしれないなと思う。未来を見通すのは難しい。

◆ ルビサンプル

コンピュータサイエンスの^{しょ ぽ}初歩として必ず^{かなら とうじょう}登場するのが、^{がた}ノイマン型
コンピュータ（^{フォン ノ イ マ ン アーキテクチャ}von Neumann architecture）もしくはコンピュータの
^{だいそう ち}5大装置だ。

コンピュータサイエンスの初歩として必ず登場するのが、^{フォン ノ イ マ ン アーキテクチャ}ノイマン型
コンピュータ（von Neumann architecture）もしくはコンピュータの5
大装置だ。コンピュータは「制御」「演算」「記憶」「入力」「出力」とい
う5つの装置で構成されるという説で、^{フォン・ノイマン氏}フォン・ノイマン氏^氏がその論文
を書いたので、ノイマン型と呼ぶ。

ここまでは本文です。

コラムタイトル

ここは**囲み記事**にします。囲み記事は<div class="column">と</div>で囲んでください。ここは**囲み記事**にします。囲み記事は<div class="column">と</div>で囲んでください。ここは**囲み記事**にします。囲み記事は<div class="column">と</div>で囲んでください。

◆数式サンプル

インライン数式は $y = \frac{1}{x^2}$ こんな感じになります。

ディスプレイ数式は

$$f(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$$

となります。

$$\begin{aligned}\frac{2}{3} + \frac{1}{5} &= \frac{10}{15} + \frac{3}{15} \\ &= \frac{13}{15}\end{aligned}$$



CSS
20mm
四方



350dpiの写真



200dpiの写真



96dpiの写真

会話サンプル

なき

はてな

がお

ノーマル

えがお

ふむ

ショック

それだ

えがお

ノーマル

交互ノーマル

交互ふむ

交互えがお

交互ショック

このページが空いています！ 埋めてください！

編集コメントを
小さく表示でき
ます。

chapter1

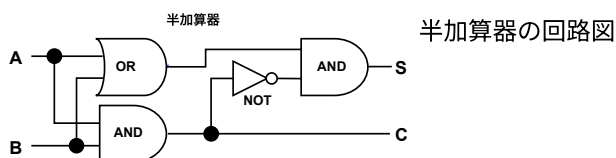
半加算器のアセンブリ実装

『Vivliostyle CSS組版入門』 サンプル



1-

半加算器の回路図



1-

回路図の解析

半加算器の回路図から読み取れる論理：

- OR ゲート: A と B の少なくとも一方が1なら1
- NOT ゲート: 入力を反転
- AND ゲート: A と B が両方1なら1

最終的に

- 出力S (和) : $A \oplus B$
- 出力C (桁上がり) : $A \text{ AND } B$

◆ アセンブリコード (x86-64 AT&T構文)

```
# データセクション
.data
input_a: .byte 0    # 入力A
input_b: .byte 0    # 入力B
output_s: .byte 0    # 出力S (和)
output_c: .byte 0    # 出力C (桁上がり)
```

```

# テキストセクション
.text
.globl _main

_main:
    # 入力値を設定 (例: A=1, B=1)
    movb $1, input_a    # A = 1
    movb $1, input_b    # B = 1

    # 半加算器の実行
    call half_adder

    # プログラム終了
    mov $60, %rax        # sys_exit
    mov $0, %rdi         # exit status
    syscall

half_adder:
    # 入力値をレジスタにロード
    movb input_a, %al    # A を al にロード
    movb input_b, %bl    # B を bl にロード

    # S = A XOR B の計算
    movb %al, %cl        # A を cl にコピー
    xorb %bl, %cl        # cl = A XOR B
    movb %cl, output_s   # 結果をSに保存

    # C = A AND B の計算
    movb %al, %dl        # A を dl にコピー
    andb %bl, %dl        # dl = A AND B
    movb %dl, output_c   # 結果をCに保存

    ret

```

1-

使用するアセンブリ命令

◆ 1. データ移動命令

- `movb`: 8ビットデータの移動
- `movb $1, %al`: 即値1をalレジスタに格納
- `movb %al, %cl`: alレジスタの値をclレジスタにコピー

◆ 2. 論理演算命令

- `xorb %bl, %cl`: XOR 演算（排他的論理和）
- `andb %bl, %cl`: AND 演算（論理積）

◆ 3. 制御命令

- `call half_adder`: 関数呼び出し
- `ret`: 関数からの復帰

1-4 真理値表

A	B	S (A XOR B)	C (A AND B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

1- レジスタの使用

- %aL, %bL: 入力値A, Bの格納
- %cL: XOR演算の結果（和S）
- %dL: AND演算の結果（桁上がりC）

1- 実行手順

1. 入力値A, Bをメモリに設定
2. half_adder関数を呼び出し
3. レジスタに入力値をロード
4. XOR演算でS（和）を計算
5. AND演算でC（桁上がり）を計算
6. 結果をメモリに保存
7. 関数から復帰

```
#include <stdio.h>

// C言語での半加算器実装（アセンブリ相当の動作）
void half_adder(int a, int b, int *s, int *c) {
    // アセンブリでの XOR 操作に相当
    *s = a ^ b; // S = A XOR B

    // アセンブリでの AND 操作に相当
    *c = a & b; // C = A AND B
}

int main() {
```

```
// 実際の計算例
printf("\n計算例:\n");
printf("=====\n");
int s, c;

half_adder(1, 1, &s, &c);
printf("1 + 1 = %d%d (2進数)\n", c, s);
printf("      = %d (10進数)\n", c * 2 + s);

half_adder(1, 0, &s, &c);
printf("1 + 0 = %d%d (2進数)\n", c, s);
printf("      = %d (10進数)\n", c * 2 + s);

return 0;
}
```

計算例:

=====

0 + 0 = 00 (2進数)

 = 0 (10進数)

0 + 1 = 01 (2進数)

 = 1 (10進数)

1 + 0 = 01 (2進数)

 = 1 (10進数)

1 + 1 = 10 (2進数)

 = 2 (10進数)